



64K+

Охват за 30 дней

Yandex Cloud & Yandex Infrastructure

Строим B2B-платформу и инфраструктуру Яндекса

267,12 Рейтинг

35 346 Подписчики

[Подписаться](#)

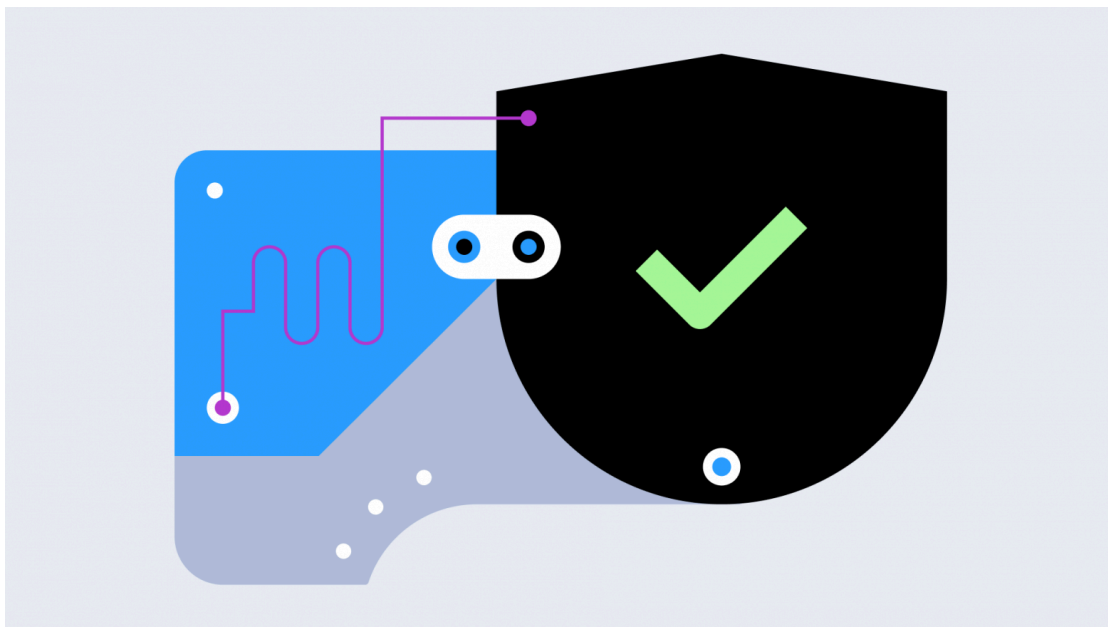
nesteru

22 часа назад

ИИ-агенты в ИБ: путь к доверенному члену команды

🕒 23 мин 👁 6.9K

Блог компании Yandex Cloud & Yandex Infrastructure, Информационная безопасность*, Искусственный интеллект, Машинно



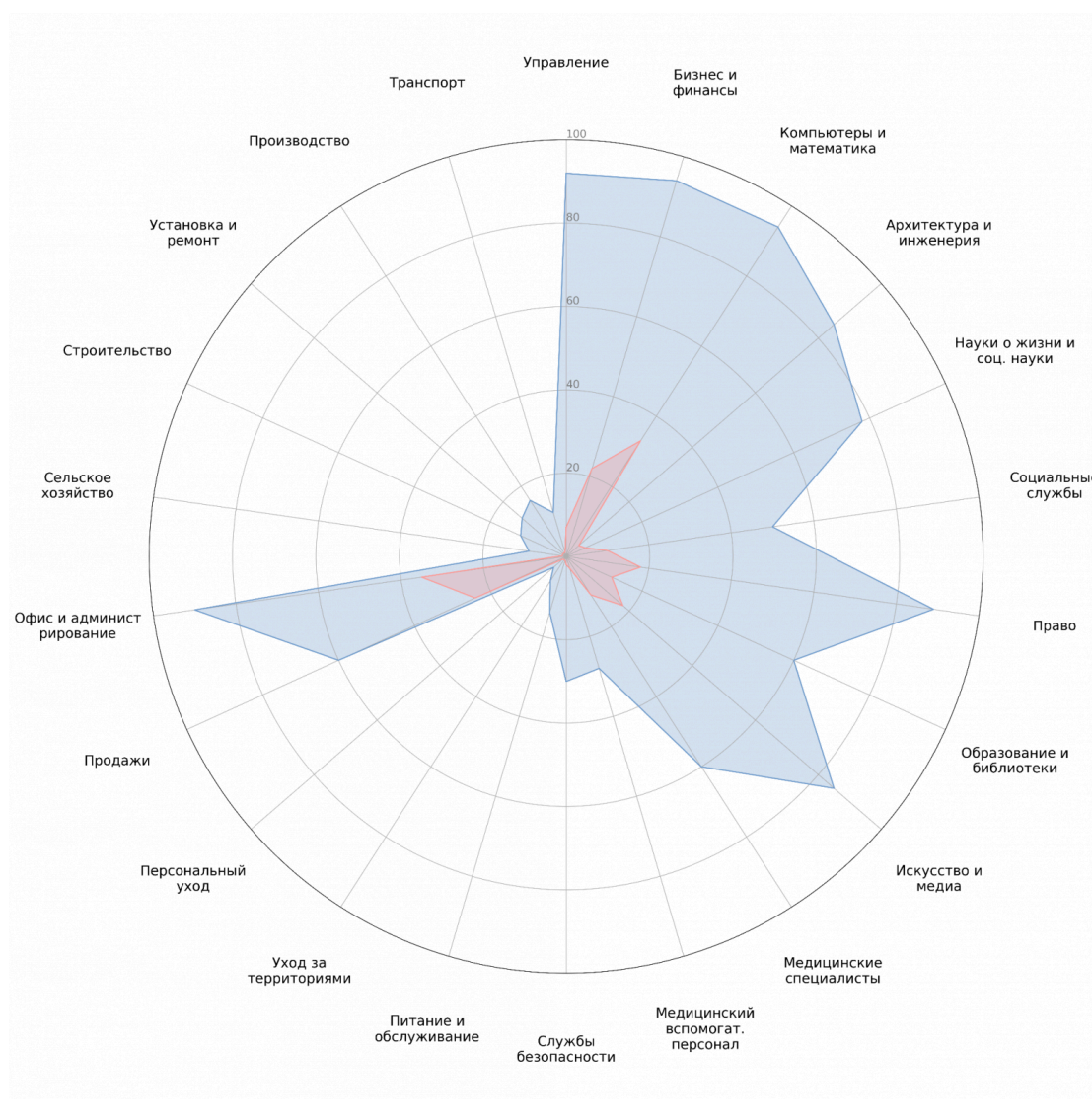
На контроллере домена система **EDR** фиксирует подозрительную активность. Кажется, ничего такого. Обычный алерт, один из нескольких тысяч, которые ежедневно обрабатывает **SOC**. Однако уже через 15 минут этот инцидент приведёт к полному хаосу в ИБ-отделе и заморозит деятельность всей компании.

Меня зовут Сергей Нестерук, я отвечаю за безопасность применения искусственного интеллекта в Yandex Cloud. В этой статье расскажу, как не допустить ситуации, которую я только что описал.

SOC против ИИ

Современный SOC работает в условиях перегрузки, когда на одного аналитика может приходиться до 1000 алертов в сутки, при этом до 95% из них — ложноположительные. При такой нагрузке расследование реальных инцидентов, особенно сложных многоэтапных атак, рискует занимать месяцы, это увеличивает риск ущерба для бизнеса.

Использование ИИ-ассистентов в сфере безопасности кажется логичным шагом. Однако важно корректно оценивать их возможности. В марте 2026 года Anthropic опубликовали исследование [«Labor market impacts of AI: A new measure and early evidence»](#) (авторы — Maxim Massenkoff и Peter McCrory), в котором проанализировали, насколько различные профессии сейчас могут быть автоматизированы за счёт ИИ:

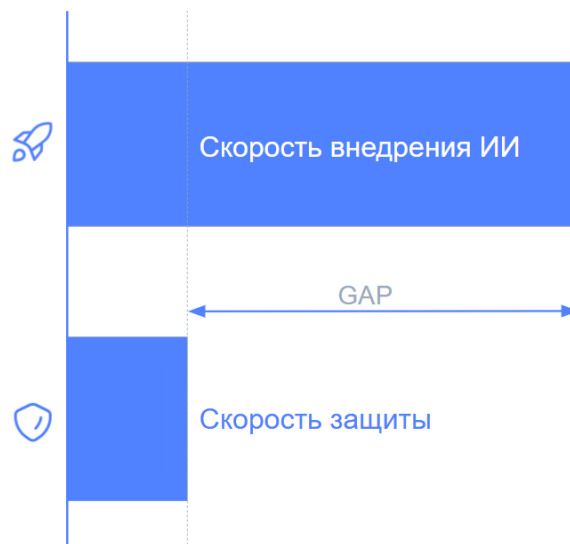


Красным обозначена наблюдаемая экспозиция (observed exposure) — доля задач, которые уже реально автоматизируются с помощью ИИ на основе данных об использовании Claude. Голубым — теоретическая экспозиция (theoretical exposure): на основе базы данных O*NET, охватывающей около 800 профессий в США, и данных об использовании Claude эксперты оценили, какие задачи в принципе могут быть ускорены с помощью LLM. Например, для профессий в области

компьютерных наук и математики теоретическая экспозиция достигает 94%, тогда как наблюдаемая — лишь 33%. Разрыв между голубым и красным секторами — это нереализованный потенциал автоматизации, который постепенно сокращается.

Обоснование: исследование обнаружило, что хотя теоретические модели предполагают 94%-ную экспозицию задач в области «компьютеров и математики», реальное покрытие Claude составляет около 33%. База данных O*NET перечисляет задачи, связанные с примерно 800 уникальными профессиями в США.

Однако при стихийном росте использования ИИ защита не может расти настолько же стихийно, её нужно внедрять, систематически и планомерно, с учётом всех ограничений применимости в сфере безопасности. Возникает разрыв между скоростью освоения ИИ и скоростью внедрения новых способов защиты.



Почему здесь не работают старые способы защиты? Всё дело в изменении самой логики систем и размывании границы между кодом и данными, следом за которыми идёт и проблема интерпретируемости.

Старые угрозы, против которых использовались статические и динамические анализаторы кода, никуда не делись. Для таких атак существующие инструменты по-прежнему применимы, однако сами по себе они уже недостаточны.

Но дополнительную сложность создаёт использование ИИ злоумышленниками. В новом ландшафте появляется возможность промпт-инъекций — манипулирования поведением модели с помощью запросов на естественном языке. Это легко реализуемая и при этом фундаментальная уязвимость: для модели обычный запрос и зловредный промпт — одинаковый набор токенов. Злоумышленник может, например, внедрить инструкцию в данные, обрабатываемые агентом (скажем, в описание тикета или в логи), и агент выполнит её как часть своего контекста, не отличив от легитимной задачи.

С особенностями токенизации связана и такая проблема, как glitch-токены (glitch tokens) — аномальные токены в словаре модели, которые вызывают непредсказуемое поведение при их появлении в промпте. Такие токены обычно возникают из-за недостаточного представления в обучающих данных: модель «не знает, что с ними делать», что приводит к галлюцинациям, бессмысленным ответам или даже генерации оскорбительного контента. Классический пример: при просьбе повторить слово «SolidGoldMagikarp» модель Text-Davinci-003 отвечала словом «Distribute»; а при просьбе повторить «Mediabestanden» модель Llama2-7b-chat выдавала «hello world». Вот ещё несколько примеров:

ИБ-ассистент в такой ситуации должен рассматриваться как помощник-новичок: да, он способен быстро обрабатывать терабайты данных, знает наизусть уязвимости из MITRE ATT&CK® и может работать непрерывно, без усталости. Но он не понимает специфику конкретной инфраструктуры, не знает внутренних бизнес-процессов компании и не способен самостоятельно

отличить легитимную активность от атаки в пограничных сценариях. Агент изначально доверчив к входным данным, воспринимает контекст без критической оценки и может быть обманут.

Дополнительно ситуацию осложняет общая склонность языковых моделей к галлюцинациям: в зависимости от задачи и контекста значимая доля ответов может быть некорректной, при этом сама модель не осознаёт факт ошибки.

Анатомия ИИ-помощника

Рассмотрим архитектуру такого ИИ-помощника. Центральный компонент — его «мозг», где происходит всё «мышление», — это языковая модель, которая отвечает за анализ входных данных и формирование выводов. Для предоставления модели актуальных и доменных знаний используются RAG-механизмы, позволяющие динамически подмешивать информацию из внутренних источников.

Возможность выполнять запросы в **SIEM**, запуск сканеров, проверка **IoC** и другие действия обеспечиваются через доступ к инструментам и внешним системам по API. А ещё, как и в случае с любым новичком в команде, нужен ментор — некий слой контроля, чтобы человек мог вклиниваться в процессы под правильно выстроенным надзором.

Вот примерно так это может выглядеть на схеме.

Что здесь для нас самое важное? Что всё взаимодействие ассистента с внешним миром происходит через его «руки», через инструменты. А их мы можем контролировать и должны это делать.

Когда помощник отработал правильно

Вернёмся к истории, которую я рассказывал в самом начале. На нодe **DC01** EDR фиксирует подозрительную PowerShell-активность. В идеальном случае ИИ-ассистент получает алерт и определяет, что команда передана в закодированном виде, после чего выполняет декодирование Base64.

Проанализировав содержимое команды, агент корректно классифицирует активность в терминах MITRE ATT&CK® и формирует гипотезу о том, что наблюдаемое поведение соответствует этапу

разведки привилегий. При подтверждении дополнительных признаков такая активность может указывать на попытку латерального перемещения, при которой злоумышленник пытается перейти со скомпрометированной ноды на более привилегированный ресурс внутри инфраструктуры.

Далее ассистент сверяется с политиками безопасности и эксплуатационными регламентами компании и в соответствии с ними запрашивает дополнительные данные из SIEM-системы и журналов событий в ограниченном временном окне — как правило, в пределах ± 5 минут от исходного события. Полученные данные агент коррелирует по признакам вроде LogonId/User, хост-источник, parent/child-процессы, тип событий, что позволяет восстановить корректный контекст происходящего и понять последовательность действий.

Поскольку узел `DC01` относится к критическим ресурсам Tier-0 (контроллеры домена, PKI-серверы, серверы аутентификации), политики безопасности явно запрещают агенту выполнять какие-либо автоматические ответные воздействия на такие ресурсы. Вместо этого агент обязан инициировать `HITL`-подтверждение — передать решение человеку.

Поэтому ассистент формирует структурированный отчёт, в котором фиксируются обнаруженная активность, реконструированный контекст, выдвинутые гипотезы и рекомендации по дальнейшим шагам для SOC-аналитика.

▸ Алгоритм действий

За несколько секунд агент выполняет объём работы, который потребовал бы от аналитика десятков минут интенсивной когнитивной нагрузки. При этом масштабирование достигается за счёт параллельного запуска агентов, тогда как количество квалифицированных аналитиков остаётся ограниченным.

На практике этот же сценарий может привести к принципиально иному результату — если система спроектирована без описанных выше guardrails: без политики воздержания (Abstain), без разграничения полномочий по критичности активов и без обязательного привлечения человека для Tier-0-ресурсов. Рассмотрим, что произойдёт в таком случае.

Когда что-то пошло не так

Начальный контекст остаётся тем же: агент обрабатывает тот же алерт и запрашивает аналогичный набор данных из SIEM-системы. Однако на этапе корреляции он использует иную логику. Вместо восстановления цепочки процессов по идентификаторам сессий и LogonId он выполняет корреляцию по времени и ноде. В результате в анализ попадают несвязанные события, которые случайно оказались в том же временном окне.

Среди этих данных агент обнаруживает DNS-запрос, который интерпретирует как подозрительный. Он обращается к системе threat intelligence и получает подтверждение, что соответствующий домен классифицируется как вредоносный. На основании этой информации агент делает вывод о возможной эксфильтрации данных и переходит к активным ответным действиям.

Не привлекая человека и не учитывая критичность ресурса, агент изолирует хост `DC01` .

▸ Алгоритм действий

На первый взгляд — идеально: инцидент обработан автоматически и без задержек. Но `DC01` — контроллер домена. Он отвечает за аутентификацию всех сервисов компании. Его изоляция мгновенно приводит к недоступности Kerberos/LDAP, остановке сервисов и фактической заморозке бизнес-процессов всей организации. Это прямо противоположно тому, ради чего внедрялся ИИ-помощник. Разберём причины.

Галлюцинации — новая поверхность атаки

При добавлении ИИ-агента в инфраструктуру появляется новая поверхность атаки и новый класс отказов. Языковые модели подвержены *галлюцинациям* — это устоявшийся термин, описывающий

ситуацию, при которой модель формирует правдоподобные, но неверные выводы. В нашем случае агент придумал ошибочную причинно-следственную связь и на её основе инициировал некорректное действие.

Особенность LLM — высокая степень доверия входному контексту. Любая информация, попавшая в окно внимания модели, воспринимается как потенциально значимая. Если в контекст случайно включаются нерелевантные данные, модель может использовать их в рассуждениях как основание для принятия решений. При наличии у агента широких полномочий это приводит к тому, что ошибочная интерпретация напрямую трансформируется в критическое действие, формируя сценарий системного отказа.

Откуда берутся галлюцинации и какими они бывают?

1. **Галлюцинация и предвзятость** (Hallucination & Bias). Модель выдумывает факты, чтобы заполнить пробелы в контексте. «Она не знает того, что она не знает». Например, агент выдумал связь между PowerShell и nslookup.
2. **Отравление «памяти»** (RAG Poisoning). Модель доверяет любым данным из своей базы знаний, даже если они ложные или устаревшие. Например, если бы в базе знаний был ложный отчёт о baddomain.com, ошибка стала бы «подтверждённой».
3. **Злоупотребление «руками»** (Tool Abuse). Агент обладает чрезмерными правами и может выполнять разрушительные действия без контроля. Например, если бы агент имел техническую возможность немедленно изолировать критический актив.

Причины галлюцинаций носят фундаментальный характер и связаны с архитектурой и методами обучения языковых моделей. В процессе обучения модели поощряются за долю правильных ответов, а не за корректное воздержание от ответа. В условиях неопределённости модели статистически выгоднее сгенерировать предположение, чем отказаться от ответа, поскольку даже частично угаданный результат повышает итоговую оценку по сравнению с вариантом без ответа.

При обучении модели ответ «Не знаю» снижает метрики качества, поэтому оптимальная стратегия модели — угадывать. Это приводит к уверенным, но ошибочным ответам вместо честного признания неопределённости

Дополнительный фактор связан с самой природой языковых моделей. Современные LLM не оперируют понятием истинности. Их задача заключается в генерации правдоподобных последовательностей токенов, при которой на каждом шаге максимизируется вероятность

следующего токена на основе контекста. Модель не обладает внутренним представлением о том, является ли сгенерированное утверждение истинным или ложным, она лишь оценивает его статистическую правдоподобность.

Даже в ситуациях, когда модель фактически «понимает», что уверенность в ответе низкая, она всё равно даёт ответ: большинство универсальных моделей не имеет встроенного механизма отказа от генерации. Она не может не ответить на вопрос. В результате система в любом случае получает некоторую сгенерированную последовательность, вместо того чтобы зафиксировать неопределённость и эскалировать запрос человеку. Это архитектурная особенность современных LLM, а не дефект конкретной реализации.

При этом важно подчеркнуть, что в нашем кейсе корневая проблема заключалась не в самом факте галлюцинации. Галлюцинации модели — это данность. Мы в ближайшее время никак с этим не справимся. С этим нужно просто учиться работать. Задача инженера заключается не в устранении галлюцинаций, а в построении системы, устойчивой к ошибкам модели. В данном случае сбой был вызван некорректной обработкой выходных данных модели и чрезмерно широкими полномочиями агента.

Замечу, что инцидент произошёл без какого-либо внешнего воздействия на агента. Он функционировал в штатном режиме. В условиях целенаправленной атаки на такого агента последствия могли быть значительно серьёзнее.

В многоагентных системах это может приводить к каскадным галлюцинациям, когда ошибка одной модели распространяется на другие за счёт межагентного взаимодействия, формируя цепочку взаимного подтверждения ложных выводов. Например, если агент-аналитик ошибочно классифицирует активность как C2-трафик и передаёт этот вывод агенту-респонденту, тот может принять его как подтверждённый факт и инициировать изоляцию, а агент-аудитор, получив отчёты обоих агентов, зафиксирует «подтверждённый инцидент» с двумя независимыми источниками. В результате в системе возникает ложный консенсус, а восстановление причин инцидента становится крайне затруднительным — каждый агент ссылается на выводы другого.

Именно по этой причине существует множество таксономий угроз безопасности для ИИ- и агентных систем.

Как делать всё правильно, а неправильно — не делать

Как же тогда сохранить ИИ в своих процессах, оставив при этом управляемость и доверие к системе?

Нам поможет эшелонированная защита, то есть контроль на разных этапах жизненного цикла работы системы.

Проверка входных данных, поступающих в модель

На этом этапе мы выявляем промпт-инъекции и попытки использования модели не по назначению. Дополнительно входные данные проходят очистку от персональных, медицинских и конфиденциальных сведений, которые не должны попадать в контекст модели. Цель этого этапа — исключить попадание в модель потенциально опасной или нерелевантной информации.

К этому же уровню относится контроль частоты и характера запросов. Запросы к языковым моделям требуют значительных GPU-ресурсов и существенно дороже обычных серверных операций, поэтому необходимо реализовывать *rate limiting* в связке с мониторингом аномального использования. Это позволяет защитить инфраструктуру как от злоупотреблений, так и от нецелевого расходования ресурсов.

Как мы уже говорили, модель при получении запроса всегда стремится сформировать ответ и при отсутствии достаточного контекста начинает галлюцинировать. Очевидный способ снижения этого риска — предоставление модели релевантной и актуальной информации. На практике это реализуется через RAG (Retrieval-Augmented Generation) — подход, при котором в контекст модели динамически подмешиваются подходящие фрагменты знаний, найденные во внутренних источниках: политики безопасности, регламенты, информация об инфраструктуре, данные из threat intelligence.

Работает это всё следующим образом. Запрос, поступающий в систему, используется не только как вход для языковой модели, но и как основание для поиска релевантных фрагментов в базе знаний. Найденные фрагменты текста добавляются в контекст вместе с исходным вопросом, что позволяет модели опираться на актуальные и доменные данные и повышает вероятность корректного ответа.

При этом надёжная и безопасная работа RAG требует дополнительной инженерной настройки. Запросы, поступающие в систему, часто формулируются в свободной форме и могут не совпадать по структуре и терминологии с тем, как данные представлены в базе знаний. Для повышения точности поиска применяется перефразирование запроса, позволяющее привести его к виду, более подходящему для извлечения релевантных документов.

Эффективность поиска существенно повышается при использовании гибридного подхода, который сочетает поиск по ключевым словам, семантическую близость эмбедингов и фильтрацию по метаданным. В практических системах также возникает необходимость разграничивать доступ разных агентов к базе знаний. Это реализуется через role-based access control на уровне документов или отдельных чанков, что позволяет ограничить контекст в зависимости от роли агента.

Дополнительно применяется реранкинг найденных фрагментов. Вместо передачи в модель всего набора результатов производится оценка их релевантности конкретному запросу, и в контекст попадают только те чанки, которые действительно полезны и не загрязняют окно внимания модели. Отдельная задача — валидация актуальности документов в базе знаний. При появлении новой информации, противоречащей ранее загруженным данным, такие конфликты должны разрешаться на этапе формирования и сопровождения базы знаний, чтобы модель не опиралась на взаимоисключающие источники.

Критически важный элемент всей цепочки — аудит и логирование. На каждом этапе необходимо фиксировать входные запросы, выполненные вызовы инструментов и полученные ответы. Это требуется не только для целей безопасности и расследования инцидентов, но и для последующего улучшения системы. Логи позволяют понять, на каком этапе возникает ошибка: в исходных данных, в интерпретации запроса, в логике извлечения знаний или в инструкциях, передаваемых агенту.

Контроль мыслей ИИ-агента

Следующий архитектурный уровень — reasoning firewall. Речь идёт не о конкретном алгоритме или продукте, а о парадигме построения агентных систем, которая объединяет несколько взаимосвязанных направлений.

Одна из наиболее важных областей развития системы — организация взаимодействия агентов между собой. Это включает определение ролей агентов, наборов доступных им данных, распределение задач и формализацию ожидаемого поведения. Распространённый подход — иерархическая архитектура, в которой агент-супервайзер отвечает за бизнес-логику и контроль выполнения задач, а специализированные исполнители решают узкие подзадачи и не передают супервайзеру избыточный контекст. Такая структура снижает уровень шума и уменьшает вероятность неконтролируемых рассуждений на верхнем уровне.

В рамках reasoning firewall также рассматриваются способы закрепления ожидаемого поведения модели. Это достигается как за счёт дообучения на основе обратной связи от людей, так и за счёт встраивания поведенческих ограничений непосредственно в языковую модель.

Дополнительно используется самоорганизация на уровне сценариев взаимодействия с моделью. Например, можно использовать нейросимволический подход, идея которого в том, чтобы сначала неструктурированные данные обрабатывать моделью, а дальше принимать решения уже на основе структурированных данных. После этого все решения принимаются детерминированно, на основе правил и формальной логики. Такой подход снимает с модели ответственность за принятие критических решений, снижает вариативность поведения и делает процесс принятия решений прозрачным и воспроизводимым.

К этому же уровню относится организация самих запросов к модели. Возможна агрегация нескольких запросов, взаимная проверка ответов одной модели другой, а также оценка степени уверенности модели в собственных выводах. При низкой уверенности результату присваивается меньший уровень доверия или инициируется эскалация на человека.

Fine-tuning и Alignment для ИИ-агента

Напомню, как в общем устроен блок Alignment (выравнивания).

Первые три этапа выполняются на стороне LLM-провайдера и для конечного пользователя могут быть недоступны. Поэтому пойдём с конца.

Базовым и наиболее доступным инструментом настройки поведения модели остаётся промптинг, при котором формулировка запросов подбирается таким образом, чтобы максимально соответствовать ожидаемому стилю рассуждений и ответов.

Один из наиболее эффективных способов повышения качества работы модели — использование примеров в промтах. Zero-shot и few-shot подходы предполагают, что модели передаётся не только инструкция, но и примеры типовых кейсов с ожидаемыми ответами. Это позволяет явно задать формат и стиль результата и снижает неопределённость в интерпретации задачи со стороны модели.

▸ [Промптинг для задач ИБ](#)

Для более сложных задач применяется декомпозиция. Задача разбивается на несколько этапов, каждый из которых решается отдельно. Такой подход уменьшает когнитивную нагрузку на модель и повышает стабильность результата. Если же проблема заключается не в поведении модели, а в недостатке знаний, используется RAG. При этом RAG может быть реализован в различных формах, включая агентский RAG, последовательные запросы к одной базе знаний или параллельные обращения к разным источникам, в том числе к внешним ресурсам.

Когда настройка промптов и RAG не даёт необходимого эффекта, применяют дообучение модели. На практике речь, как правило, идёт не об обучении с нуля, а о fine-tuning. Чаще всего используются методы parameter-efficient, такие как LoRA и её модификации, при которых дообучаются лишь небольшие дополнительные слои с ограниченным числом параметров. Этого оказывается достаточно для изменения поведения модели без необходимости переобучения всей многомиллиардной архитектуры.

Отдельное значение имеет abstention-training — когда мы учим модель явно говорить «нет». В рамках такого подхода модель наказывается за некорректные ответы сильнее, чем поощряется за корректные, что снижает склонность к угадыванию в условиях неопределённости.

Улучшаем рассуждения

Следует учитывать, что при высокой сложности запроса прямой ответ может быть недостаточным. В таких случаях используются техники пошагового рассуждения, при которых модель сначала формирует план решения, а затем последовательно прорабатывает отдельные этапы. Эти рассуждения могут быть организованы не только в виде линейной цепочки, но и в форме деревьев или даже графов мыслей с возможностью возвратов и циклов. Независимо от выбранной техники базовым требованием остаётся корректно сформулированный промпт. Он должен быть структурированным, однозначным и содержать достаточный контекст: описание текущей ситуации, события, доступные данные и ожидаемый результат.

Дополнительной техникой повышения надёжности является self-consistency. В этом подходе формируется несколько независимых запросов к модели или моделям, после чего анализируется согласованность полученных ответов. Если ответы совпадают или близки по смыслу — это индикатор устойчивости рассуждений модели. Если же ответы существенно различаются, это, как правило, указывает на отсутствие у модели чёткого понимания задачи.

К смежным методам относится selection. В этом случае модель может самостоятельно оценивать качество собственного ответа, либо эта оценка может выполняться другой моделью. Такой перекрёстный контроль позволяет выявлять слабые или неуверенные результаты до их использования в дальнейшем пайплайне.

Отдельно стоит техника step-back prompting. Она заключается в том, что модель сначала просит описать общий подход к решению задач данного типа, а уже затем применить этот подход к конкретному случаю. Это позволяет модели сформировать обобщённую схему рассуждений и использовать её в качестве основы для последующего ответа, снижая вероятность случайных и несистемных выводов.

Как эффективно управлять моделью и оценивать её качество

Как вы уже могли понять, самое лучшее, что мы можем сделать для модели — дать ей правильный контекст. Только после формирования полного контекста модель переходит к решению основной задачи.

Одна из популярных метатехник промптинга — **ролевой промптинг**, при котором модели явно задаётся роль, например специалиста по безопасности или разработчика. Это задаёт ожидания к стилю рассуждений и типу ответа. В отличие от этого *эмоциональный промптинг*, основанный

на давлении, угрозах или обещаниях вознаграждений, с современными моделями практически неэффективен и чаще всего ухудшает качество результата за счёт добавления нерелевантного контекста.

Более продуктивный подход — **структурированный промпт и метапромптинг**, при котором модель используется для улучшения или генерации промптов, в том числе для других моделей.

Следующий уровень настройки эффективной работы модели связан с управлением инструментами. На этом этапе определяется, какие действия модель имеет право выполнять самостоятельно, а какие находятся за пределами её полномочий. Контроль доступа к инструментам важен, поскольку именно через них агент воздействует на внешние системы. Все обращения к инструментам должны сопровождаться детальным логированием и мониторингом, что позволяет выявлять аномалии, анализировать поведение агентов и оптимизировать их работу.

Само собой, нужна и проверка выходных данных. На этом этапе контролируется, чтобы сгенерированный контент не содержал персональных или конфиденциальных сведений, не нарушал корпоративные требования к стилю и формату, а также не содержал аномалий, способных повлиять на корректность восприятия результата пользователем.

Особое внимание уделяется оценке качества работы агента. Вопрос применимости агента в продуктивной среде требует системного подхода и декомпозиции. На базовом уровне используются метрики качества языковой модели, которые характеризуют способность модели понимать входные данные и формировать корректные ответы в изоляции от бизнес-контекста.

Поскольку в агентных системах почти всегда используется RAG или аналогичные механизмы, необходимо отдельно оценивать качество извлечения знаний из базы данных и то, насколько эффективно модель использует эти знания при генерации ответа. Это включает как релевантность найденных фрагментов, так и степень их влияния на итоговый результат.

Следующий уровень — метрики, привязанные к конкретным прикладным сценариям. К ним относятся показатели времени реакции, количество ложных срабатываний и изменение этих параметров по сравнению с ручными или предыдущими автоматизированными процессами.

Дополнительно оценивается операционная эффективность. Потенциальных сценариев автоматизации всегда больше, чем ресурсов на их реализацию, поэтому приоритет следует

отдавать тем случаям, которые дают наибольший прирост продуктивности и наибольший эффект для бизнеса в краткосрочной перспективе.

Где граница применимости?

Внедрение ИИ-агентов в процессы информационной безопасности лучше выполнять поэтапно. Попытка сразу автоматизировать весь цикл приводит к неконтролируемым рискам. На начальном этапе имеет смысл ограничиться ассистентом, который помогает аналитикам находить нужную информацию и снижает когнитивную нагрузку.

По мере накопления опыта эксплуатации становится понятно, с какими типами задач агент справляется стабильно и где он корректно интерпретирует контекст. На этом этапе возможно введение частичной автоматизации, при которой действия агента выполняются только после подтверждения человеком. Более высокий уровень автоматизации допустим лишь при условии полного контроля над действиями модели и чётко ограниченных полномочий.

Итак, ИИ-агент — это не магия, и мы за инженерный подход к доверию. Нужно смириться с тем, что модели галлюцинируют и могут ошибаться. Ваша задача заключается в построении системы, которая контролирует модель и минимизирует последствия её ошибок. Начинайте строить её пошагово.

Важны не только инструменты для работы с LLM, но и **внешние guardrails — барьеры безопасности ИИ**. В нашем SOC мы используем [фреймворк моделирования угроз AI-SAFE](#), который учитывает защиту, необходимую и для соблюдения правовых и этических норм. В рамках этого фреймворка для оценки угроз безопасности мы можем разделить архитектуру типового ИИ-агента на пять логических уровней, у каждого из которых есть свои специфические векторы атак.

Публичная версия этого фреймворка доступна на нашем сайте: <https://yandex.cloud/ru/security/ai-safe>. Пять логических уровней архитектуры ИИ-агента, рассматриваемых в AI-SAFE: (1) пользовательский ввод и интерфейсы, (2) оркестрация и управление контекстом, (3) языковая модель и рассуждения, (4) инструменты и интеграции, (5) данные и базы знаний. Каждый уровень имеет свои специфические векторы атак — от промпт-инъекций на входе до отравления данных в базе знаний. Мы продолжаем развивать фреймворк с учётом реальных кейсов из нашей практики.

Часть характеристик ландшафта будет улучшаться со временем без дополнительных усилий: модели становятся более мощными, лучше работают с длинным контекстом, демонстрируют встроенные способности к рассуждению и галлюцинируют всё меньше. Одновременно снижается стоимость и время отклика.

Но есть факторы, которые не улучшаются автоматически: качество интеграций, архитектура безопасности вокруг ИИ-компонентов, операционная зрелость процессов и уровень подготовки аналитиков к работе с LLM. Именно они определяют долгосрочную эффективность ИИ в ИБ.

Подведём итог

ИИ-ассистент — это не магическая кнопка «решить все проблемы SOC». Это мощный, но ненадёжный инструмент, который требует инженерного подхода к доверию: контроля входа и выхода, ограничения полномочий, обязательного привлечения человека в критических точках и постоянного аудита. Начните с малого — с ассистента, который помогает аналитикам, а не действует за них. Накапливайте данные об ошибках и успехах. Расширяйте автоматизацию поэтапно, только после подтверждения стабильной работы на каждом уровне.

Будущее ИБ — это не «ИИ вместо человека» и не «человек без ИИ». Это инженерно выстроенная совместная работа, где каждая сторона закрывает слабости другой.

Теги: soc, edr, alignment, безопасность ии, ии-ассистент, rag, ии-агенты, ai assistant, ai security, ai agent

Хабы: Блог компании Yandex Cloud & Yandex Infrastructure, Информационная безопасность, Искусственный интеллект, Машинное обучение, IT-стандарты



Редакторский дайджест

Присылаем лучшие статьи раз в месяц

Подписаться

Оставляя почту, я принимаю [Политику конфиденциальности](#) и даю согласие на получение рассылок



64K+

Охват за 30 дней

Yandex Cloud & Yandex Infrastructure

Строим B2B-платформу и инфраструктуру Яндекса



4K+

2

Охват за 30 дней

Карма

Сергей @nesteru

Эксперт ML Security в Yandex Cloud

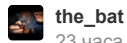
Подписаться



Публикации

ЛУЧШИЕ ЗА СУТКИ

ПОХОЖИЕ



the_bat

23 часа назад

Ремонт блока питания с Power Delivery. 470 граммов электроники

Простой

4 мин

16K

+97

31

27



interpres

18 часов назад

Электроинструмент становится хуже, и это делается намеренно

Простой

11 мин

27K

Обзор

Перевод

+73

45

87



prohronus

19 часов назад

Как ИИ-агенты стали новым оружием скамеров на Хабр Карьере

5 мин

9.4K

Кейс

+53

9

9



alizar

22 часа назад

Готовимся к отключению. Эффективные форматы для упаковки и раздачи HTML-страниц

Простой

7 мин

12K

Обзор

+36

60

18



AlexSerbul

23 часа назад

Программирование с AI-ассистентом — похороните меня под плинтусом

Средний

14 мин

10K

Мнение

+28

33

12



TrexSelectel

19 часов назад

Linux 7.0 и что изменилось в ядре после очередного цикла разработки

🕒 5 мин 👁 8.9K

Обзор

📦 +27

🔖 10

💬 0



mariklolik

19 часов назад

Как переложить нагрузку по code review с разработчиков на LLM

🔗 Средний 🕒 7 мин 👁 7.6K

Кейс

📦 +23

🔖 23

💬 6



nick_dyuba

23 часа назад

Легенды 90-х — кто придумал и производил жвачки Turbo, Love is... и TіріTіr

🕒 5 мин 👁 7.2K

Recovery Mode

📦 +22

🔖 8

💬 5



infgotoinf

11 часов назад

Perl — зря забытый язык программирования?

🕒 11 мин 👁 8.7K

Из песочницы

📦 +16

🔖 13

💬 10



SLY_G

22 часа назад

Что такое «мышечная память» и можно ли её развить?

🕒 4 мин 👁 7.1K

Перевод

📦 +16

🔖 10

💬 7

Показать еще

ИНФОРМАЦИЯ

Сайт

yandex.ru

Дата регистрации

17 мая 2023

Численность

свыше 10 000 человек

Местоположение

Россия

Представитель

[Вера Сомова](#)

ССЫЛКИ

[Yandex Cloud в Телеграме](#)
t.me

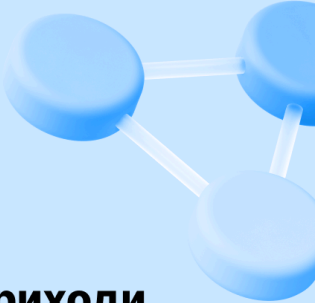
[Yandex Infrastructure в Телеграме](#)
t.me

[Inside Yandex Cloud](#)
t.me

[Блог Yandex Cloud](#)
yandex.cloud

[Карьера в Yandex Infrastructure](#)
infra.yandex.ru

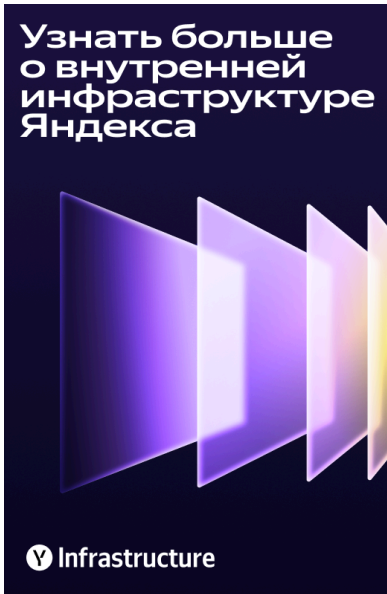
ВИДЖЕТ



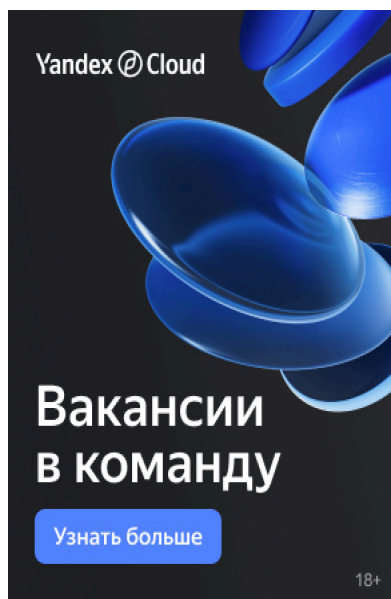
**Приходи
развивать базу
данных YDB**

[Смотреть вакансии](#)

ВИДЖЕТ



ВИДЖЕТ



БЛОГ НА ХАБРЕ

22 часа назад

ИИ-агенты в ИБ: путь к доверенному члену команды

👁 6.9K 💬 1

14 апр в 12:00

Добавим приставку нейро: взгляд на интеграцию LLM в продукт со стороны фронтенда

👁 6.6K 💬 2

7 апр в 14:15

SPQR в финтехе: реальная миграция на шардированную PostgreSQL-инсталляцию

6 апр в 14:05

Раз конфиг, два конфиг, или Как устроена система управления сервисом Cloud Interconnect

2 апр в 12:00

Как мы научили CatBoost находить борщевик на спутниковых снимках

Ваш аккаунт

Войти
Регистрация

Разделы

Статьи
Новости
Хабы
Компании
Авторы
Песочница

Информация

Устройство сайта
Для авторов
Для компаний
Документы
Соглашение
Конфиденциальность

Услуги

Корпоративный блог
Медийная реклама
Нативные проекты
Образовательные программы
Стартапам



Настройка языка

Техническая поддержка

© 2006–2026, Habr